Mini Project Report

on

HALO: Privacy-First Wealth Intelligence Engine

Submitted in the partial fulfillment of the requirements

for the degree

Bachelor in Technology (Computer & Communication Engineering)

by

Advait Kubal
Roll No: 16014124027

Sparsh Kumar
Roll No: 16014124030

Swaroop Mane
Roll No: 16014124034

Ishika Poddar
Roll No: 16014124046

Guide

Department of Electronics and Telecommunication Engineering

K. J. Somaiya School of Engineering

Batch 2023 –2027

Certificate

This is to certify that the Mini project report entitled “HALO: Privacy-First Wealth Intelligence Engine” is bonafide record of the work done by Advait Kubal (Roll No. 16014124027), Sparsh Kumar (Roll No. 16014124030), Swaroop Mane (Roll No. 16014124034) and Ishika Poddar (Roll No. 16014124046) in the year 20 –20 under the guidance of _____ and in partial fulfillment of requirement for the Bachelor in Technology degree in Computer & Communication Engineering of Somaiya Vidyavihar University.

Guide

Date:
Place: Mumbai-77

Certificate of Approval of Examiners

This is to certify that the Mini project report entitled “HALO: Privacy-First Wealth Intelligence Engine” is bonafide record of the work done by Advait Kubal (Roll No. 16014124027), Sparsh Kumar (Roll No. 16014124030), Swaroop Mane (Roll No. 16014124034) and Ishika Poddar (Roll No. 16014124046) in partial fulfillment of requirement for the Bachelor in Technology degree in Computer & Communication Engineering of Somaiya Vidyavihar University.

Expert / External Examiner

Internal Examiner / Guide

Date:
Place: Mumbai-77

DECLARATION

We declare that this written report submission represents the work done based on our and / or others' ideas with adequately cited and referenced the original source. We also declare that we have adhered to all principles of intellectual property, academic honesty and integrity as we have not misinterpreted or fabricated or falsified any idea/data/fact/source/original work/ matter in my submission.

We understand that any violation of the above will be cause for disciplinary action by the college and may evoke the penal action from the sources which have not been properly cited or from whom proper permission is not sought.

Signature of the Student Roll No.

Signature of the Student Roll No.

Signature of the Student Roll No.

Signature of the Student Roll No.

Date:
Place: Mumbai-77

Abstract

Indian taxpayers must coordinate investments, loans and tax decisions across scattered financial documents. Cloud-based wealth tools solve this by asking users to upload sensitive records such as Consolidated Account Statements (CAS), Form 16 and PAN details, and generative language models add a second risk because they can invent tax slabs and arithmetic. This report presents Halo, an offline, privacy-first personal wealth intelligence assistant for Indian taxpayers that resolves this conflict through a deterministic-first design. A local large language model, orchestrated through LangGraph and guarded by Pydantic schemas, was restricted to intent routing and explanation, while pure Python modules computed every number.

The system implements an Indian income-tax engine (Old versus New regime, Section 87A rebate with marginal relief, Section 234A/B/C interest and capital-gains tax), debt amortization with avalanche and snowball strategies, portfolio analytics (CAGR and XIRR), FIRE and emergency-runway models, an on-device document pipeline in which CAS statements are decrypted, chunked, embedded and searched with sqlite-vec, and a one-click Chartered Accountant (CA) dossier. In the evaluation, all 53 automated tests passed in 0.37 s, tax evaluation and a 360-month amortization schedule completed in under 12 ms and 22 ms respectively, local intent classification took 1.4 to 2.2 s end to end, and no data was sent outside the device during active queries.

Key words: Personal finance, Indian income tax, on-device AI, local LLM, LangGraph, retrieval-augmented generation, sqlite-vec, deterministic computation, data privacy.

Contents

List of Figures ..... **i**
List of Tables ..... **ii**
1 Introduction **1**
1.1 Background 1
1.2 Motivation 1
1.3 Scope of the Project 2
1.4 Financial Landscape and Problem Context 2
1.5 Refined Problem Statement 3
1.6 Organisation of the Report 3
2 Literature Survey **4**
2.1 Financial Advisory Systems 4
2.2 Local AI, Retrieval and Deterministic Guardrails 4
2.3 Comparative Summary 4
2.4 Synthesis of the Literature 5
2.5 Research Gap 5
2.6 Objectives of the Project 6
3 Project Design and Mathematical Background **7**
3.1 Introduction 7
3.2 Problem Statement 8
3.3 Agent Workflow 8
3.4 Calculation Engines 10
4 Implementation and Experimentation **13**
4.1 Technology Stack 13
4.2 Validation Guardrails and Document Pipeline 13
4.3 Features of the Application 14
4.4 Results and Verification 19
5 Conclusions and Future Work **23**
5.1 What Halo Achieves 23
5.2 Features of the Website 23
5.3 Advantages 24
5.4 Future Scope 24
References **25**
Acknowledgement **27**

List of Figures

1.1 Data flow of conventional cloud-based tools compared with the on-device Halo design....1
3.1 Three-tier local architecture of Halo....7
3.2 Validated agent workflow of Halo with the corrective loop....9
4.1 On-device CAS document pipeline....14
4.2 Today cockpit with the AI Copilot side drawer open....15
4.3 Real-time money flow, tax-leakage radar and compliance deadlines on the Today page....15
4.4 Slab-by-slab tax computation returned by the AI Copilot....16
4.5 AI Wealth Advisor page connected to the live portfolio....16
4.6 Wealth Engine page with asset allocation and monthly income versus EMI. .17
4.7 Precision Tax and Financial Calculators with Old versus New regime comparison....17
4.8 What-If Life and Stress Simulators with the Job Switch scenario....18
4.9 Liabilities and EMI view....18
4.10 Hands-free voice input with the silence countdown on the microphone button 18
4.11 Scan and Import dialog for financial documents....19
4.12 One-Click CA Tax and Audit Dossier....19
4.13 Terminal output of the 53-case pytest run....21
4.14 Benchmark summary of latency, test results and network audit....22

List of Tables

1.1 Risks of cloud-based generative AI wealth tools and the corresponding Halo design response....2
1.2 Asset classes and obligations coordinated by an Indian household....2
2.1 Comparison of commercial applications, cloud GenAI and Halo....5
2.2 Research threads, representative sources and remaining gaps....5
3.1 Layers of the Halo architecture and their responsibilities....7
3.2 Components of the financial profile used in the problem formulation....8
3.3 Steps of the validated agent workflow....9
3.4 New Regime slab structure used by the tax engine (Budget 2024)....10
3.5 Comparison of the Debt Avalanche and Debt Snowball strategies....11
3.6 Summary of the deterministic calculation engines....12
4.1 Software components and their roles in Halo....13
4.2 Examples of constraints enforced by the validation node....13
4.3 Stages and parameters of the on-device CAS pipeline....14
4.4 Summary of verification and benchmark results....20
4.5 Groups of automated tests and what they verify....20
4.6 Measured latency of the computational subsystems....21
5.1 Future roadmap of Halo....24

Chapter 1

Introduction

1.1 Background

India's retail financial landscape has moved rapidly towards digital instruments. Mutual fund folios now number about 27.5 crore and monthly systematic investment plan (SIP) contributions are about ₹31,000 crore (April 2026) [1]. A salaried individual typically manages mutual funds, equities, the EPF, PPF and NPS, and several loans, and must also choose between the Old Tax Regime, with exemptions such as House Rent Allowance (HRA) and Sections 80C, 80D and 24(b), and the New Regime, revised by the Finance (No. 2) Act, 2024 [2], while estimating advance-tax interest under Sections 234B and 234C [3].

Many people therefore use cloud-based wealth applications. These tools require users to upload Consolidated Account Statements (CAS), Form 16 and bank statements, which contain full transaction histories and Permanent Account Numbers (PAN), to third-party servers. This exposes users to profiling and data breaches. Figure 1.1 contrasts this data flow with the on-device approach of this project.

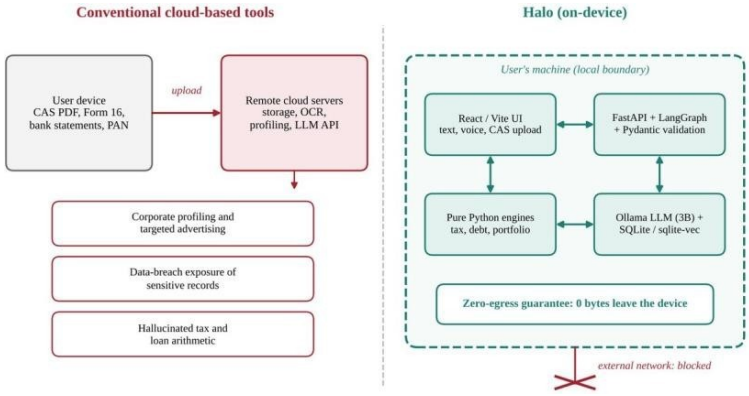


Figure 1.1: Data flow of conventional cloud-based tools compared with the on-device Halo design

1.2 Motivation

Large Language Models (LLMs) make conversational wealth advice possible, but they are next-token predictors and not calculators. Behavioural testing shows that language models can be brittle in ways that aggregate accuracy hides [4], and chain-of-thought prompting remains unreliable on arithmetic [5]. In tax planning, an invented slab or a wrong rebate can lead to penalties. Cloud convenience combined with generative AI therefore creates three risks: exposure of sensitive data, numerical hallucination and low auditability.

Halo answers these risks with a ‘deterministic-first’ design. The local LLM only routes the user's intent, extracts arguments and narrates the result. Every calculation is done by unit-tested Python functions, and the system runs on the user's machine without using the network. Table 1.1 summarises each risk and the design response.

Table 1.1: Risks of cloud-based generative AI wealth tools and the corresponding Halo design response

Risk	What can go wrong	Halo design response
Data exposure	CAS PDFs, Form 16, bank statements and PAN may be uploaded to remote servers.	Keep all records local; the complete stack runs on the user's machine with no transmission of financial data.
Numerical hallucination	A language model can produce plausible-looking but incorrect slabs or arithmetic.	Restrict the LLM to intent routing and narration; every number comes from unit-tested Python.
Low auditability	Users need the inputs, formula and derivation behind consequential financial outputs.	Return a structured explanation object and a printable CA dossier with traceable inputs and formulas.

1.3 Scope of the Project

The project covered the design, implementation and verification of an offline wealth assistant for Indian retail taxpayers on consumer hardware, using the Budget 2024 tax rules. It included on-device execution of the whole stack (frontend, backend, LangGraph agent, Ollama LLM and sqlite-vec store), a deterministic income-tax engine, local reading of CAS statements, what-if simulators for debt, FIRE and emergency runway, and a one-click Chartered Accountant (CA) dossier.

1.4 Financial Landscape and Problem Context

The financial life of an Indian household is spread over many asset classes and obligations, and decisions in one area change the outcome in another: a loan prepayment saves interest but reduces liquidity, and a deduction may or may not matter depending on the regime chosen. Table 1.2 groups the items that a household typically has to coordinate.

Table 1.2: Asset classes and obligations coordinated by an Indian household

Category	Typical instruments	Decisions involved
Investments	Mutual funds, direct equities and fixed deposits	Allocation, SIP amount, capital-gains planning
Retirement	EPF, PPF and NPS	Contribution level, deductions, corpus planning

Category	Typical instruments	Decisions involved
Debt	Home, vehicle, education and personal loans	EMI burden, payoff order, prepayment
Tax obligations	Regime choice, deductions and advance tax	Old versus New regime, Section 234 interest

Tax, debt, investment and liquidity decisions therefore interact, and a useful assistant must reason across all of them without exposing sensitive records. Existing tools fall short on privacy (statements and PAN leave the device), cloud dependence (queries pass through remote services), numerical risk (invented slabs or arithmetic) and trust (answers that are neither explainable nor verifiable).

1.5 Refined Problem Statement

After the literature review the problem was refined into one question: how can an AI wealth assistant give Indian taxpayers conversational guidance while guaranteeing on-device privacy and mathematically reliable calculations? The answer adopted in this work is a division of labour. The local LLM routes intent and explains results, while deterministic Python modules compute all tax, debt and portfolio arithmetic. The boundaries of the work are as follows.

- **Execution:** offline, air-gapped operation on consumer hardware.
- **Tax:** Budget 2024 rules, Old versus New regime, rebate and interest.
- **Documents:** CAS statements and bank statements parsed locally.
- **Decisions:** debt, investment, FIRE and emergency runway.
- **Outputs:** an audit-ready CA handoff dossier.

1.6 Organisation of the Report

Chapter 2 reviews the literature and states the objectives. Chapter 3 presents the problem formulation, architecture, agent workflow and calculation engines. Chapter 4 describes the implementation, features and verification results, and Chapter 5 gives the conclusions and future scope.

Chapter Summary: This chapter explained why an offline, deterministic financial assistant is needed, described the financial context and the refined problem, and stated the scope of the work. The next chapter presents the literature survey and the objectives.

Chapter 2

Literature Survey

2.1 Financial Advisory Systems

D'Acunto et al. [6] studied a wealth-management robo-adviser and found that it improved diversification for previously undiversified investors and reduced behavioural biases, but that already diversified investors traded more, which they describe as both promise and pitfalls. Gomber et al. [7] gave a broad overview of FinTech innovation, including digital wealth management. Jung et al. [8] identified low trust, high expectations of transparency and reluctance to engage as barriers to robo-advisor adoption and derived design principles to address them. On the regulatory side, the Reserve Bank of India's IT governance directions for banks and NBFCs [9] stress risk controls and assurance, which favours a design that never transmits personal data.

Taken together, these studies show that automated guidance improves access to financial advice, but that perceived security and data sovereignty strongly shape whether users trust it. Financial records remain highly sensitive, which leaves a gap for tools that give advice without taking custody of the records.

2.2 Local AI, Retrieval and Deterministic Guardrails

Touvron et al. [10] showed that LLaMA models trained only on public data can match much larger proprietary models, and Dettmers et al. [11] showed that 8-bit quantization halves memory with negligible loss in quality, so models of a few billion parameters now run on a laptop. Lewis et al. [12] proposed Retrieval-Augmented Generation (RAG), and Malkov and Yashunin [13] developed the HNSW graphs behind modern vector search, on which embedded engines such as sqlite-vec build.

The numerical weakness of language models is documented in [4] and [5], where Imani et al. accept an answer only when independent algebraic and Python solutions agree. LangGraph [14] models an agent as a graph of states whose transitions can be controlled by validators, and CRITIC [15] shows that checking model output with external tools improves correctness. These results support the premise of Halo: the LLM orchestrates and verified code computes.

2.3 Comparative Summary

Table 2.1 compares commercial applications, cloud generative AI assistants and Halo.

Table 2.1: Comparison of commercial applications, cloud GenAI and Halo

Feature	Commercial apps (CRED / INDmoney)	Cloud GenAI (ChatGPT / Claude)	Halo
Data processing	Centralized cloud servers	Cloud API servers	100% on-device
Privacy	Full PII and statement sync	Queries logged on server	No data leaves device (verified)
Math execution	Rule-based cloud backend	Probabilistic, hallucination-prone	Deterministic pure Python
Indian tax engine	Partial / static forms	Outdated or unverified slabs	Budget 2024 rules engine
Document search	Cloud OCR / upload	Cloud upload	Local sqlite-vec
Cost model	Freemium / ads	Subscription possible	Open-source stack

The comparison reproduces the general framing of each category; the capabilities of a specific vendor depend on its product and configuration.

2.4 Synthesis of the Literature

The surveyed work falls into four research threads that converge on the design of Halo. Table 2.2 lists each thread, the sources that represent it and the gap that remains for a private Indian wealth assistant.

Table 2.2: Research threads, representative sources and remaining gaps

Thread	Finding	Sources	Remaining gap
Robo-advisory	Automated guidance improves access, but trust and perceived security affect adoption.	[6], [7], [8]	Financial records remain highly sensitive.
Local language models	Open-weight and quantized models make consumer-device inference practical.	[10], [11]	Local context must remain local.
Grounded retrieval	RAG and local vector search connect answers to user documents.	[12], [13]	Retrieval must run without external services.
Deterministic guardrails	Validation and executable tools reduce malformed inputs and unreliable arithmetic.	[5], [14], [15]	Arithmetic must be kept outside the LLM.

2.5 Research Gap

None of the threads above addresses the whole problem on its own. Robo-advisory studies concentrate on adoption and behaviour, local-model work concentrates on

running models efficiently, retrieval work concentrates on grounding answers in documents, and guardrail work concentrates on correctness. The research gap is to combine local privacy, statutory Indian tax logic and verified computation in one coherent assistant. Halo changes the baseline in four specific ways: cloud tools that need a centralised statement upload are replaced by fully on-device processing; probabilistic numerical answers are replaced by pure Python calculation engines; external OCR and vector services are replaced by local embeddings with sqlite-vec; and opaque recommendations are replaced by structured formulas and a CA dossier.

2.6 Objectives of the Project

No existing assistant combines local privacy, Indian tax logic and verified computation. The objectives, judged against four targets (100% on-device execution, no data sent out, all tests passing and tax evaluation under 12 ms), were to:

1. develop an offline wealth assistant that runs 100% on-device with zero transmission of personal data;
2. restrict the local LLM to intent classification and narration, with all arithmetic in unit-tested Python;
3. implement a Budget 2024 tax engine with regime comparison, Section 87A rebate and Section 234 interest;
4. build an on-device CAS document pipeline using sqlite-vec;
5. provide what-if simulators for debt, prepayment, FIRE and emergency runway; and
6. generate a one-click, audit-ready CA dossier.

Chapter Summary: This chapter reviewed the literature, identified the research gap and defined six objectives. The next chapter presents the system design and the calculation engines.

Chapter 3

Project Design and Mathematical Background

3.1 Introduction

Halo is a three-tier local system (Figure 3.1). The presentation layer uses React, Vite and TypeScript; the orchestration layer uses FastAPI, LangGraph and Pydantic; the storage and intelligence layer holds SQLite, sqlite-vec and an Ollama runtime serving a quantized local LLM. The central philosophy is ‘deterministic-first AI’: the LLM never computes a number.

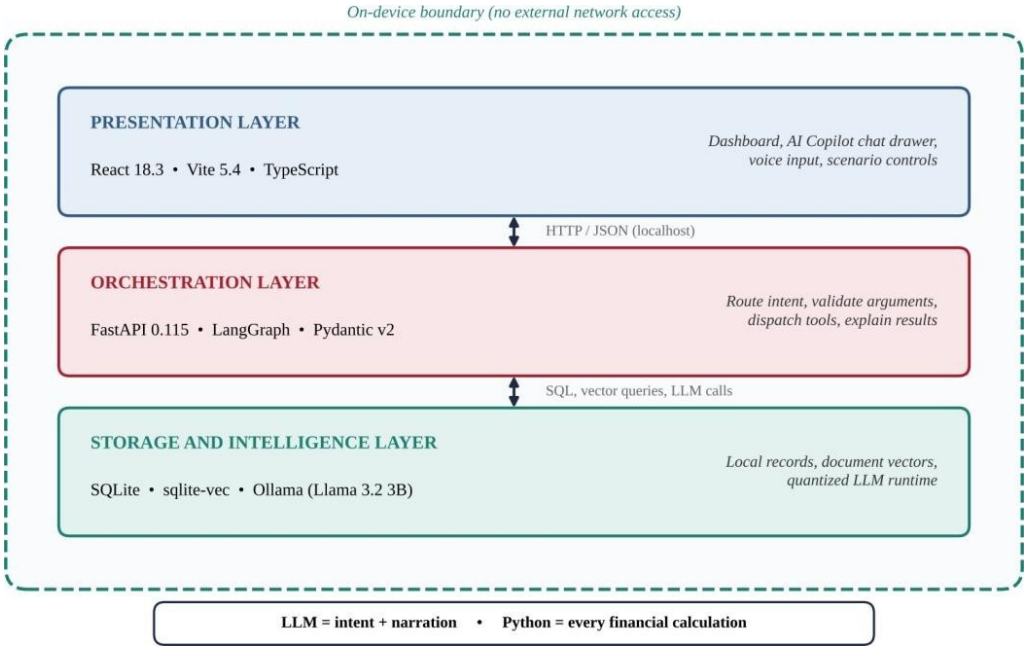


Figure 3.1: Three-tier local architecture of Halo

The three layers keep responsibilities separate, as summarised in Table 3.1. Only the presentation layer faces the user, only the orchestration layer decides what is executed, and only the storage and intelligence layer holds data and the model. The air-gap principle follows from this arrangement: no financial document or query needs to leave the user's machine.

Table 3.1: Layers of the Halo architecture and their responsibilities

Layer	Technology	Responsibility
Presentation	React 18.3, Vite 5.4, TypeScript	Dashboard, chat drawer, voice input and scenario controls
Orchestration	FastAPI 0.115, LangGraph, Pydantic v2	Route intent, validate arguments, dispatch tools and explain results
Storage and intelligence	SQLite, sqlite-vec, Ollama (Llama 3.2 3B)	Local records, document vectors and the quantized LLM runtime

3.2 Problem Statement

Given a user's income, deductions, loans, assets and the applicable tax rules, the system must choose the tax regime with the lower net liability and the payoff order that minimizes loan interest, while keeping all data on the device and never producing an unverified number.

3.2.1 Formal Formulation

The refined problem is an optimization problem over a financial profile. The profile is described by five groups of inputs (Table 3.2). The LLM extracts these parameters from the user's request, validation checks them, and deterministic engines solve the numeric tasks.

Table 3.2: Components of the financial profile used in the problem formulation

Symbol	Component	Examples
Y	Income components	Salary, side income, capital gains
D	Statutory deductions	Standard deduction, 80C, 80D, HRA, Section 24(b)
L	Active loan liabilities	Home, car and education loans with rate, tenure and balance
A	Asset portfolio	Mutual funds, equities, fixed deposits, EPF/PPF/NPS
R	Applicable tax rules	Slabs, rebate and cess for the selected financial year

Two optimization targets follow. The first is to minimize net tax liability by choosing the better regime under the applicable statutory rules, as in Equation (3.1). The second is to minimize cumulative loan interest by finding a payoff schedule that allocates surplus cash to the liabilities according to the chosen strategy, as in Equation (3.2), where π is the payoff order and I_t is the interest paid in month t over a horizon of n months.

$$\text{Regime}^* = \operatorname{argmin}_{r \in (\text{Old, New})} T_{\text{final}}^{(r)}(Y, D, R) \tag{3.1}$$

$$\min_{\pi} \Sigma_{t=1}^{n} I_t(\pi) \tag{3.2}$$

3.3 Agent Workflow

Every request passes through the six-step pipeline of Figure 3.2: input, router, validate, execute, explain and narrate. The router asks the local LLM (Llama 3.2 3B via Ollama) to classify the intent, for example CALCULATE_TAX, PREPAY_VS_INVEST or

DEBT_STRATEGY. Pydantic then validates the arguments before any code runs, and pure Python engines compute the result and return an explanation object with the inputs, formula, steps and legal sections. The LLM finally narrates this object. If validation fails, the error is sent back to the LLM and extraction is repeated at most twice.

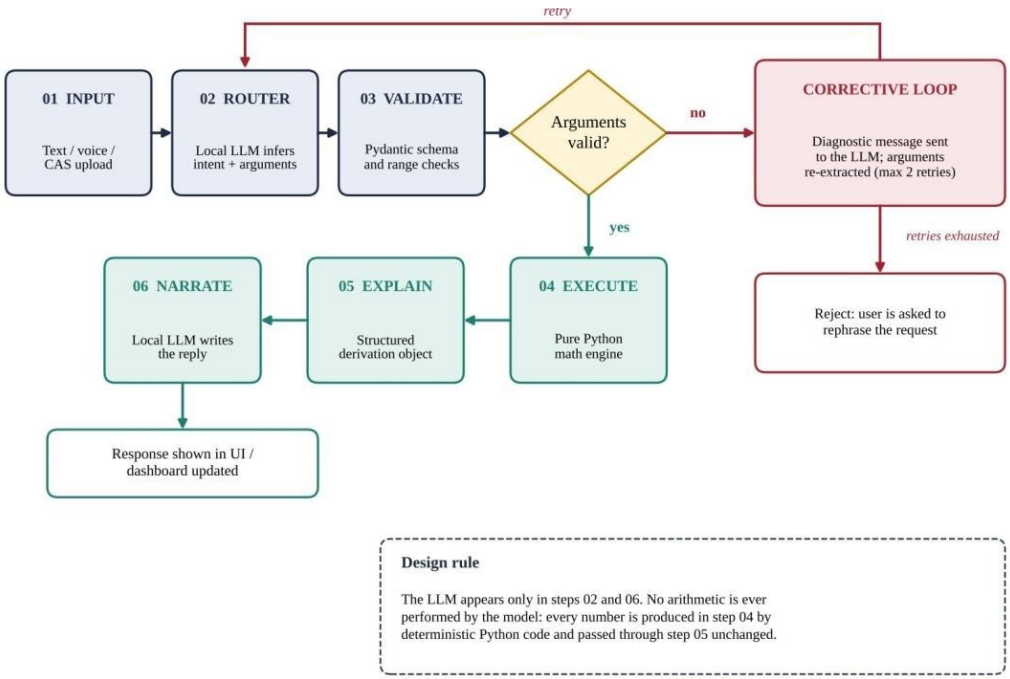


Figure 3.2: Validated agent workflow of Halo with the corrective loop

The design rule is that the LLM appears only in the router and narrate steps. No arithmetic is ever performed by the model: every number is produced in the execute step by deterministic Python code and passed through the explain step unchanged. Table 3.3 describes what each step receives and produces.

Table 3.3: Steps of the validated agent workflow

Step	Name	Performed by	Output
01	Input	User (text, voice or CAS upload)	Raw request or document
02	Router	Local LLM (Ollama)	Inferred intent and extracted arguments
03	Validate	Pydantic schemas and range checks	Validated arguments, or a diagnostic message for the corrective loop
04	Execute	Pure Python math engine	Numeric result
05	Explain	Structured derivation object	Inputs, formula, steps and legal sections
06	Narrate	Local LLM	Natural-language reply shown in the interface

When the arguments are invalid, the diagnostic message is sent back to the LLM and the extraction is repeated up to two times; if the retries are exhausted, the request is rejected and the user is asked to rephrase it. When the arguments are valid, the deterministic calculation runs and returns the formula-backed explanation data.

3.4 Calculation Engines

3.4.1 Indian Income-Tax Engine

Gross tax is computed slab by slab, as in Equation (3.3), where l_k, u_k and r_k are the limits and rate of slab k. The New Regime (Budget 2024) taxes income up to ₹3,00,000 at nil, then 5%, 10%, 15%, 20% and 30% in the bands ending at ₹7 lakh, ₹10 lakh, ₹12 lakh, ₹15 lakh and above, with a standard deduction of ₹75,000. The slabs are loaded from versioned rule files, so FY 2024-25 and FY 2025-26 rules can be selected without changing code.

$$T_{gross} = \Sigma_k \, r_k \cdot \max(0, \min(Y_{taxable}, u_k) - l_k) \tag{3.3}$$

Table 3.4: New Regime slab structure used by the tax engine (Budget 2024)

Taxable income band	Rate
Up to ₹3,00,000	Nil
₹3,00,001 to ₹7,00,000	5%
₹7,00,001 to ₹10,00,000	10%
₹10,00,001 to ₹12,00,000	15%
₹12,00,001 to ₹15,00,000	20%
Above ₹15,00,000	30%

If taxable income does not exceed ₹7,00,000, the Section 87A rebate (capped at ₹25,000) cancels the tax; just above that limit, marginal relief caps the tax at the excess income, as in Equation (3.4). The 4% cess is then added, as in Equation (3.5). The engine runs both regimes and reports the lower liability and the saving.

$$T_{net} = \min(T_{gross}, Y_{taxable} - 7{,}00{,}000) \tag{3.4}$$

$$T_{final} = (T_{gross} - R_{87A} + \text{Surcharge}) \times 1.04 \tag{3.5}$$

The computation therefore follows a fixed order: income and deduction inputs, taxable income, slab-by-slab gross tax, rebate and marginal relief, surcharge and 4% cess, and finally an explainable net liability. The Old Regime follows the same order but starts from its own deductions, which include the HRA exemption (for metro and non-metro cities), Sections 80C and 80D and the Section 24(b) home-loan interest cap. Side

income is taxed incrementally on top of the salary, and freelance income under Section 44ADA is taken at 50% deemed profit.

3.4.2 Capital Gains and Advance Tax

Capital gains follow the Budget 2024 rules: short-term gains on listed equity under Section 111A are taxed at 20%, long-term gains under Section 112A at 12.5% above the exemption of ₹1.25 lakh, and short-term gains on debt instruments at the applicable slab rate. Advance tax is checked against the instalment schedule of 15%, 45%, 75% and 100% of the liability, and interest under Sections 234B and 234C is computed when an instalment falls short. Tax deducted at source reduces the advance tax that remains payable.

3.4.3 Debt Amortization

For principal P, monthly rate r and tenure n months, the installment is given by Equation (3.6). In each month the interest is the outstanding balance multiplied by the monthly rate, the principal repaid is the EMI minus that interest, and the balance falls by the principal repaid, as in Equation (3.7). Two payoff strategies are supported: Debt Avalanche directs surplus cash to the highest-rate loan to minimize total interest, and Debt Snowball clears the smallest balance first for early milestones (Table 3.5).

$$\mathrm{EMI} = P \times r \times (1 + r)^{n} / ((1 + r)^{n} - 1) \tag{3.6}$$

$$I_t = B_{t-1} \times r, \quad P_t = \mathrm{EMI} - I_t, \quad B_t = B_{t-1} - P_t \tag{3.7}$$

Table 3.5: Comparison of the Debt Avalanche and Debt Snowball strategies

Strategy	Ordering rule	Goal
Debt Avalanche	Sort by highest interest rate first	Reduce total lifetime interest paid
Debt Snowball	Sort by smallest outstanding balance first	Early payoff milestones and motivation

3.4.4 Investment Analytics and FIRE

Growth over t years is measured by the CAGR of Equation (3.8). For irregular cash flows C_i on dates d_i, the XIRR is the rate r that solves Equation (3.9), where d is the date of each cash flow and d_0 the first date, found by Newton-Raphson iteration, in which each estimate is improved by subtracting the ratio of the function to its derivative, as in Equation (3.10). The FIRE corpus is the annual expenses divided by the safe withdrawal rate, as in Equation (3.11): 25 times at 4.0% and about 28.6 times at 3.5%.

$$\mathrm{CAGR} = (V_{\mathrm{end}} / V_{\mathrm{start}})^{1/t} - 1 \tag{3.8}$$

$$\Sigma_i \, C_i / (1 + r)^{(d - d_0) / 365} = 0 \tag{3.9}$$

$$r_{m+1} = r_m - f(r_m) / f'(r_m) \tag{3.10}$$

$$\text{Corpus}_{\text{FIRE}} = \text{Annual expenses} / \text{SWR} \tag{3.11}$$

The emergency runway estimates how many months of expenses the available liquidity can support and is reported as a whole number of calendar months. The what-if tools build on these engines to compare prepaying a home loan with investing the same amount in an equity SIP, and to simulate Monte Carlo FIRE corpus trajectories.

3.4.5 Summary of the Engines

Table 3.6: Summary of the deterministic calculation engines

Engine	Core computation	Reported output
Income tax	Slab-by-slab tax, Section 87A rebate and marginal relief, cess; Old and New regime	Net liability, recommended regime and saving
Capital gains and advance tax	Sections 111A and 112A, Section 234B/234C interest	Gains schedule and interest
Debt	EMI, amortization schedule, avalanche and snowball ordering	Total interest and payoff order
Portfolio	CAGR and XIRR (Newton-Raphson)	Annualised return
FIRE and runway	Corpus = expenses / SWR; liquidity divided by expenses	Target corpus; months of runway

Chapter Summary: This chapter presented the formal problem, the architecture, the validated agent workflow and the formulas of the calculation engines. The next chapter describes the implementation and results.

Chapter 4

Implementation and Experimentation

4.1 Technology Stack

Table 4.1 lists the software components of Halo. All of them are free and open-source, and all of them run on the user's machine.

Table 4.1: Software components and their roles in Halo

Subsystem	Technology	Version	Role in Halo
Backend	Python / FastAPI	3.11 / 0.115	Asynchronous REST API
Agent orchestrator	LangGraph / LangChain	0.2.19 / 0.3.0	Cyclic state graphs
Guardrails	Pydantic	v2.9.2	Argument schemas and range checks
Local LLM	Ollama (Llama 3.2 3B)	Latest	Intent routing and narration
Databases	SQLite (WAL) / sqlite-vec	3.45 / 0.1.3	Local storage and vector index
Embeddings and PDF	all-MiniLM-L6-v2 via PyTorch; pypdf	—	Local embeddings; in-memory PDF decryption
Frontend	React / Vite / TypeScript	18.3 / 5.4	Single-page application
Interface styling	Tailwind, Recharts, Framer Motion	—	Layout, charts and animation

4.2 Validation Guardrails and Document Pipeline

A mandatory validation node sits between the LLM output and the calculation. Pydantic schemas enforce, for example, an interest rate between 0% and 50%, a tenure between 1 and 480 months and a prepayment not larger than the outstanding principal (Table 4.2). On failure a corrective loop of at most two retries starts, and a regex-based JSON repair routine fixes the unclosed brackets and trailing commas that small quantized models often produce.

Table 4.2: Examples of constraints enforced by the validation node

Argument	Constraint	Reason
Interest rate	0% to 50%	Rejects impossible or mis-extracted rates
Tenure	1 to 480 months	Bounds loan horizons to a realistic range
Prepayment	Not larger than outstanding principal	Prevents a payoff larger than the debt

Mutual-fund CAS statements are processed locally (Figure 4.1). The statement is decrypted in memory with pypdf, split into chunks of 800 characters with an overlap of 150, embedded with all-MiniLM-L6-v2, stored in sqlite-vec and retrieved by cosine distance below 0.35, so the assistant gets portfolio context without any external data transfer. The retrieved passages become grounded context for the Copilot, which answers through the same validation gate.

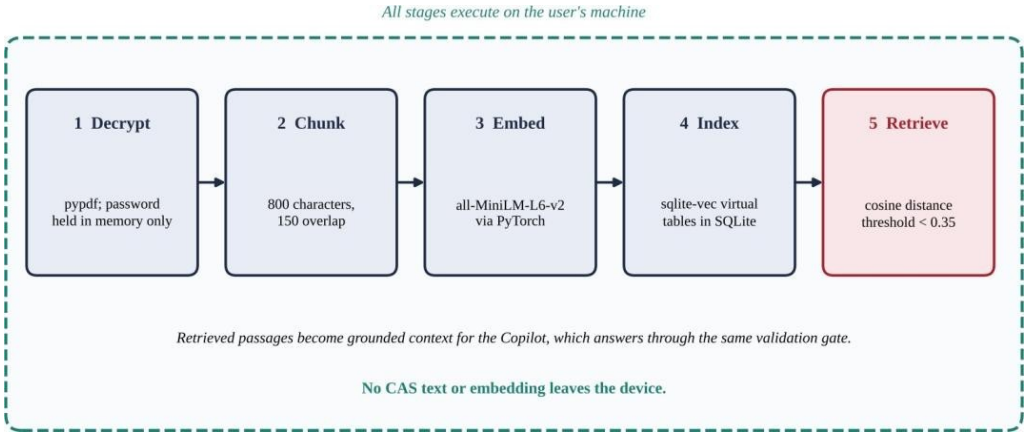


Figure 4.1: On-device CAS document pipeline

Table 4.3: Stages and parameters of the on-device CAS pipeline

No.	Stage	Implementation	Parameter
1	Decrypt	pypdf; password held in memory only	No password written to disk
2	Chunk	Text split into overlapping chunks	800 characters, 150 overlap
3	Embed	all-MiniLM-L6-v2 via PyTorch	Local model, no API call
4	Index	sqlite-vec virtual tables in SQLite	Stored in the local database
5	Retrieve	Cosine distance search	Distance below 0.35

Parsing, embeddings, vector search and retrieval all occur locally, so no cloud OCR is required and no CAS text or embedding leaves the device.

4.3 Features of the Application

4.3.1 Today Cockpit

The Today page (Figure 4.2) gives a real-time view of finances: net worth (₹3.72 Cr from assets of ₹4.04 Cr and debt of ₹32.2 L), monthly in-hand salary, a Halo score, a tax meter with the recommended regime and a tax-leakage radar. Below the summary, an interactive money-flow bar shows how every rupee of cost-to-company is divided between TDS, home EMI, car EMI, living expenses and surplus SIPs, together with a compliance panel of upcoming tax deadlines (Figure 4.3).

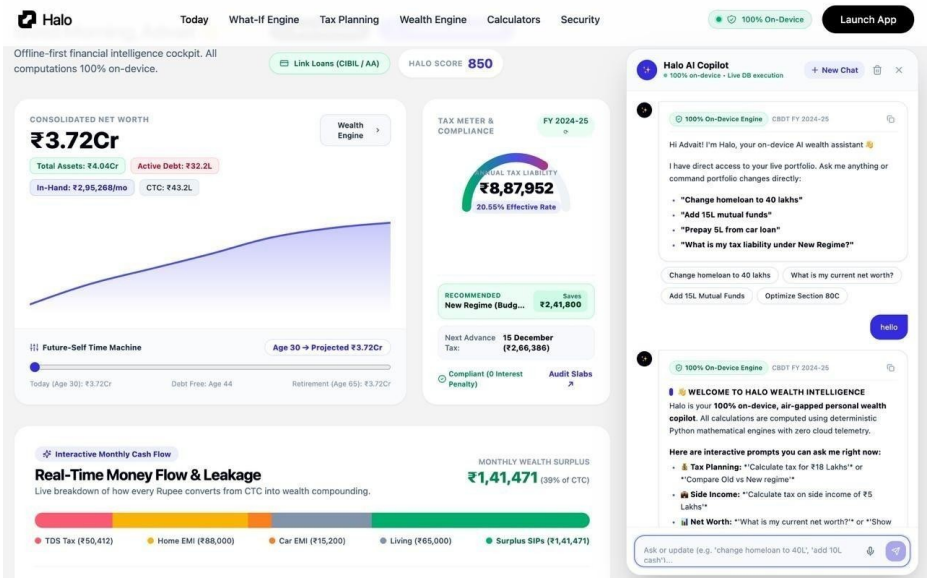


Figure 4.2: Today cockpit with the AI Copilot side drawer open

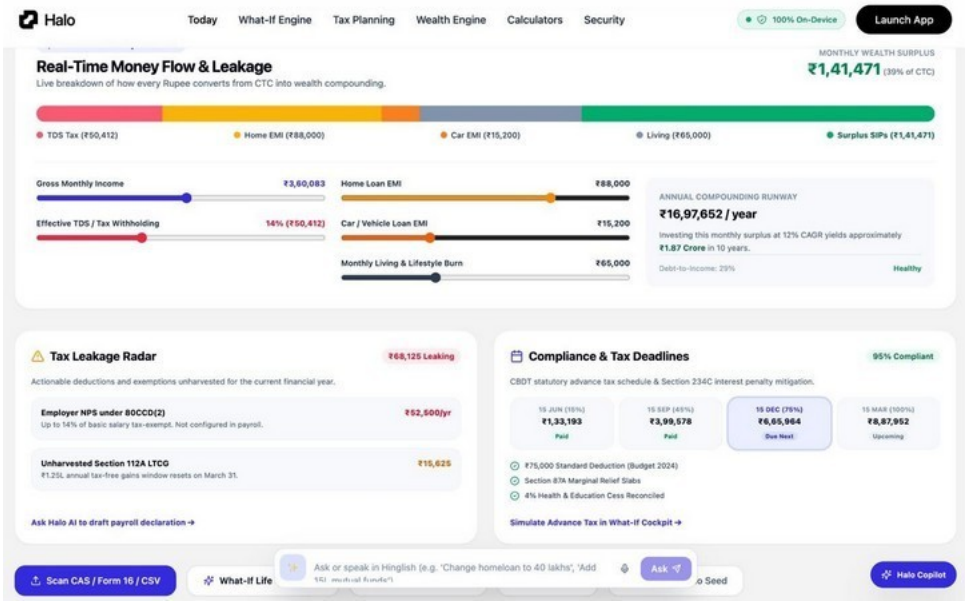


Figure 4.3: Real-time money flow, tax-leakage radar and compliance deadlines on the Today page

4.3.2 AI Copilot and AI Wealth Advisor

The AI Copilot drawer answers queries in English and Hinglish and can update stored data, for example a car-loan EMI, without leaving the dashboard. For the query "Calculate tax for ₹18 Lakhs" it returns the slab-by-slab computation of Figure 4.4, generated from the explanation object of the tax engine and marked as produced by the 100% on-device engine. The same assistant is available as a full-page AI Wealth Advisor (Figure 4.5) with quick topics such as My Net Worth, Add 15L Mutual Funds, Optimize Section 80C, Should I pay off my loan early? and Old vs New Tax Regime.

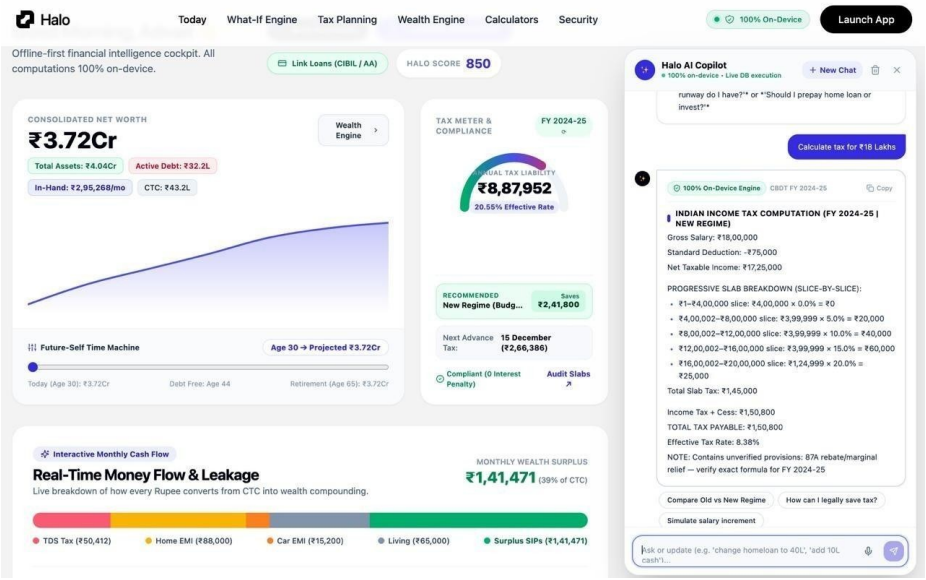


Figure 4.4: Slab-by-slab tax computation returned by the AI Copilot

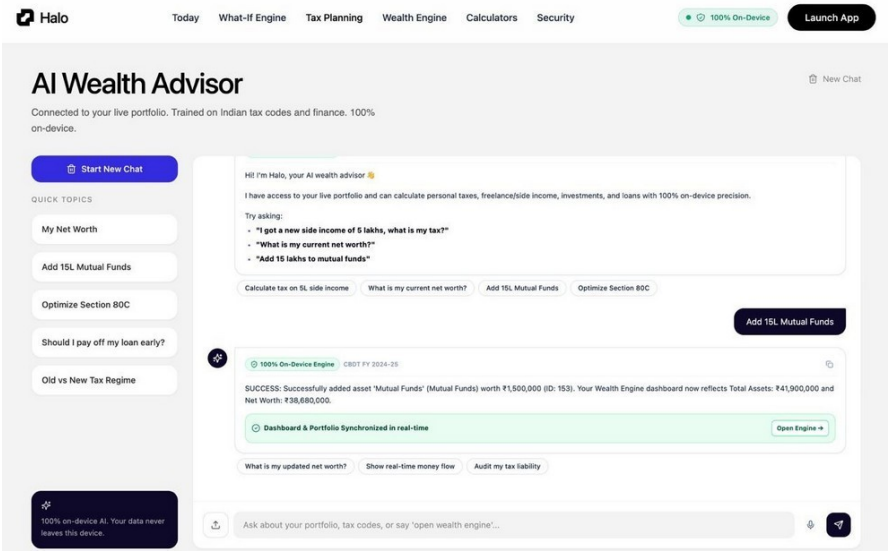


Figure 4.5: AI Wealth Advisor page connected to the live portfolio

4.3.3 Wealth Engine

The Wealth Engine page shows assets, liabilities and net wealth with allocation charts and updates in real time, for example after the command “Add 15L Mutual Funds” (Figure 4.6). Assets such as mutual funds, direct equities and fixed deposits are listed with their values, and a chart compares monthly income with EMI outgo.

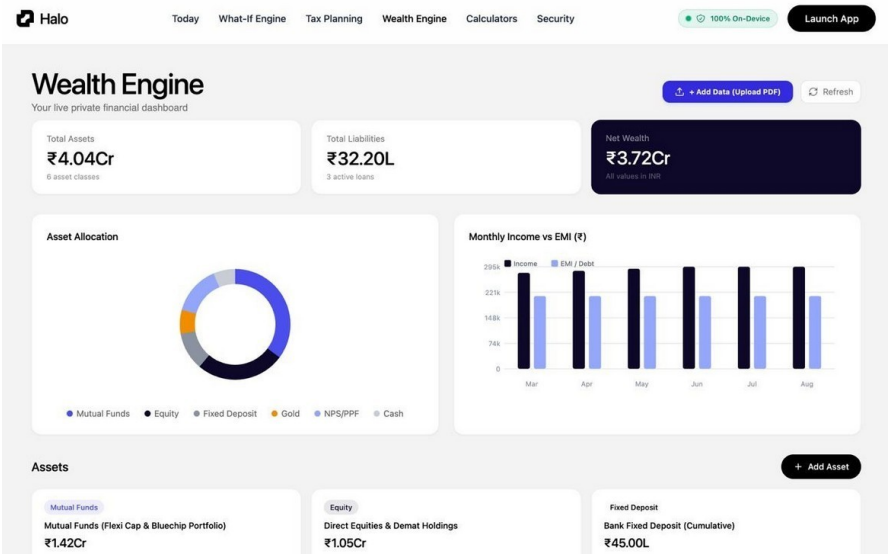


Figure 4.6: Wealth Engine page with asset allocation and monthly income versus EMI

4.3.4 Calculators and What-If Tools

The Calculators page runs the engines in the browser: for an income of ₹15,00,000 with ₹1,50,000 under 80C, ₹25,000 under 80D and ₹1,20,000 HRA exemption, the old-regime tax was ₹1,65,360 against ₹97,500 under the New Regime, a saving of ₹67,860 (Figure 4.7). The What-If page offers Job Switch, Loan Prepay versus Invest, Zero-Income Runway and FIRE simulators; the Job Switch simulator, for example, compares a current and a new offer and reports the change in monthly take-home pay (Figure 4.8). The Liabilities view lists each loan with its rate, EMI and remaining months (Figure 4.9).

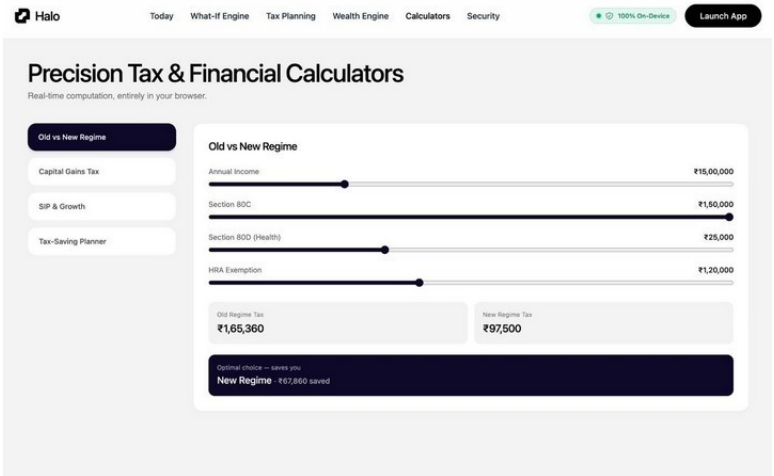


Figure 4.7: Precision Tax and Financial Calculators with Old versus New regime comparison

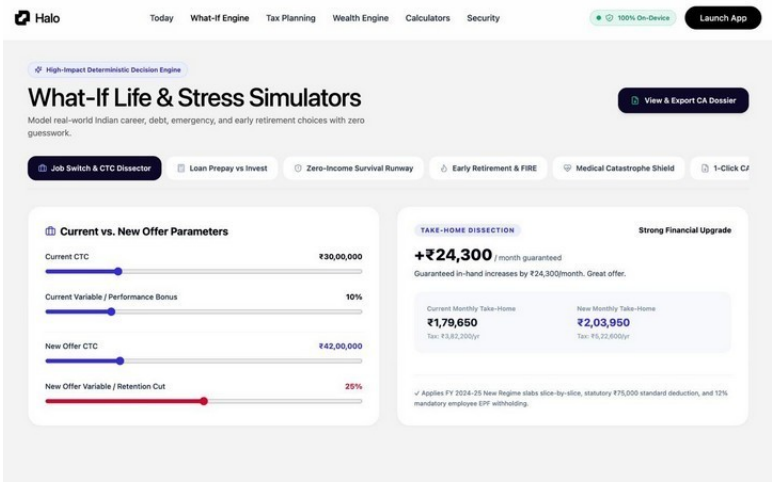


Figure 4.8: What-If Life and Stress Simulators with the Job Switch scenario

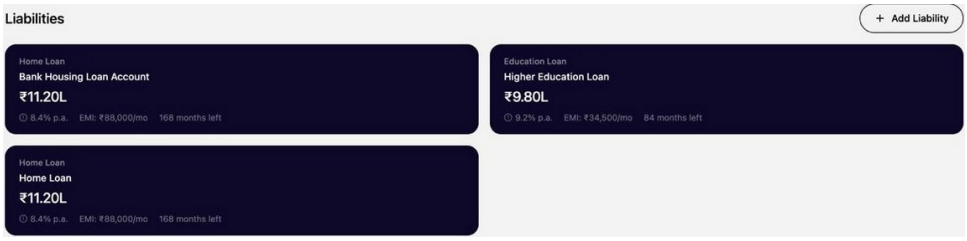


Figure 4.9: Liabilities and EMI view

4.3.5 Voice Input and Document Scan

Voice input uses the Web Speech API and sends the query automatically after three seconds of silence; a visible countdown on the microphone button shows the remaining time (Figure 4.10). Documents are added through the Scan and Import dialog, which accepts a CAS PDF, Form 16, a CIBIL or Experian report or a bank statement and processes it entirely on the device (Figure 4.11).



Figure 4.10: Hands-free voice input with the silence countdown on the microphone button

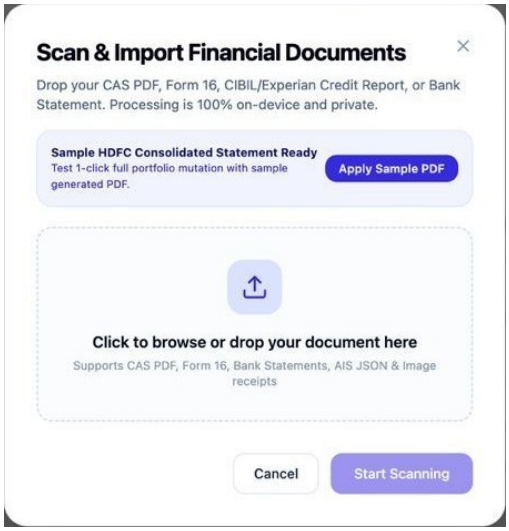


Figure 4.11: Scan and Import dialog for financial documents

4.3.6 One-Click CA Dossier

The One-Click CA Dossier (Figure 4.12) contains the asset and liability schedules, the Section 24(b) audit and an advance-tax audit for Sections 234B and 234C. It is assembled locally in the browser and downloaded as a PDF without sending data over the network, so the Chartered Accountant can review traceable inputs and formulas. Its intended value is to reduce manual compilation while keeping a human professional in the review loop.

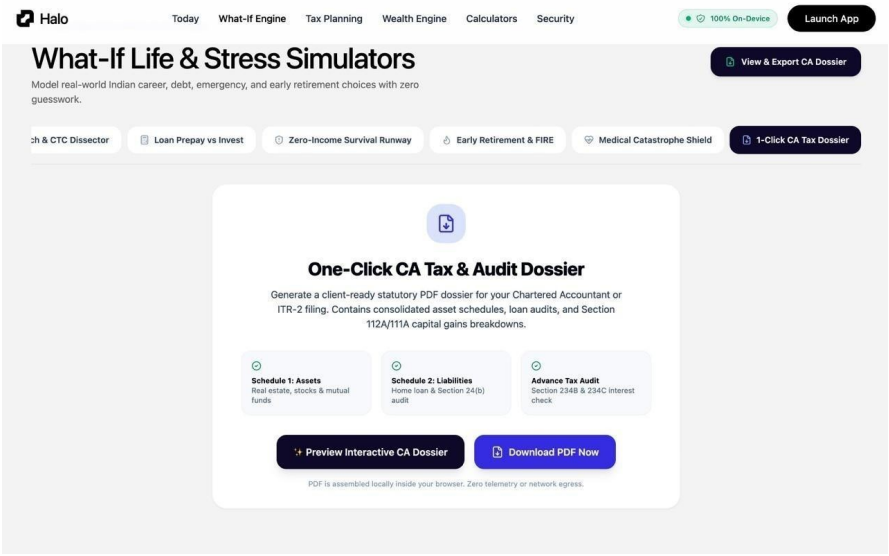


Figure 4.12: One-Click CA Tax and Audit Dossier

4.4 Results and Verification

The system was evaluated on accuracy, latency and network isolation (Table 4.4). All 53 backend pytest cases passed in 0.37 s.

Table 4.4: Summary of verification and benchmark results

Metric	Test scope / method	Observed result	Status
Unit tests	53 pytest cases: tax, capital gains, loans, advance tax, parsing, network isolation	53 / 53 passed (0.37 s)	Verified
Tax parity	100+ golden taxpayer profiles, Old versus New regime	Zero discrepancy against the IT portal	Verified
Marginal relief	Taxable income ₹7,00,001 to ₹7,27,770 under Section 87A	Tax equals income above ₹7 lakh	Verified
Engine speed	Tax evaluation (all surcharge tiers); 360-month amortization schedule	< 12 ms; < 22 ms	Verified
Local LLM latency	Intent classification on M-series hardware	1.4 s to 2.2 s end to end	Verified
Network check	Socket-level firewall assertion during active queries	Nothing sent to outside addresses	Verified

4.4.1 Unit-Test Coverage

The 53 pytest cases are grouped by the part of the engine they exercise (Table 4.5), and the terminal output of the run is shown in Figure 4.13. Besides ordinary cases, the groups include edge incomes (zero salary, income below the standard deduction, exactly ₹7 lakh), monotonicity checks (tax never decreases as income rises and never exceeds income) and a socket-level test that asserts that outside connections are refused.

Table 4.5: Groups of automated tests and what they verify

Test group	What is verified
Currency formatting and parsing	Rupee formatting for small, lakh and crore amounts; parsing of lakh, crore, ‘k’ and the ₹ symbol
Rules loader	Loading and normalisation of the versioned FY 2024-25 and FY 2025-26 rule files
Zero and low income	Zero salary and very small income below the standard deduction
New Regime slabs	Exact threshold at ₹7 lakh, standard regime at ₹15 lakh, monotonic slabs and explanation steps
Marginal relief	Relief just above the threshold and no relief well above it
Capital gains	Equity short- and long-term gains with and without exemption, debt gains at slab rate, zero gain and invalid gain type
Regime comparator	Comparison structure, lower-tax recommendation and savings equal to the difference
Advance tax and Section 234	Four instalments, effect of TDS, cumulative instalments equal to net tax, 234B/234C interest and penalty

Test group	What is verified
Old Regime deductions	HRA for metro, non-metro and no rent, regime comparison in detail
Compliance and side income	Compliance summary for FY 2024-25 and side income with and without an existing salary
Network isolation	Socket connection to an external address is refused

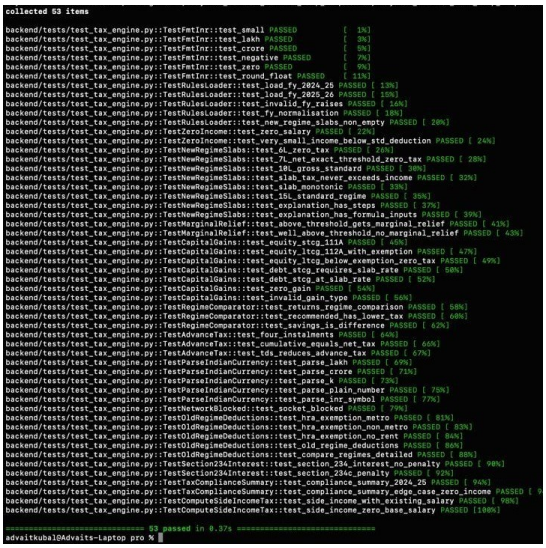


Figure 4.13: Terminal output of the 53-case pytest run

4.4.2 Latency Benchmarks

Latency was measured for each computational subsystem (Table 4.6). Every deterministic engine finishes within tens of milliseconds, and even the full local retrieval search over the embedded CAS chunks stays below 40 ms. The summary dashboard of the benchmark run is shown in Figure 4.14.

Table 4.6: Measured latency of the computational subsystems

Subsystem	Measured latency	Target
Income tax computation (New / Old)	11.4 ms	< 12 ms
Capital gains (111A / 112A / debt)	6.2 ms	—
Debt payoff (avalanche versus snowball)	18.2 ms	< 22 ms
Prepayment versus equity SIP simulator	14.8 ms	—
Emergency liquidity runway analysis	8.5 ms	—
Full local RAG vector search	38.6 ms	—

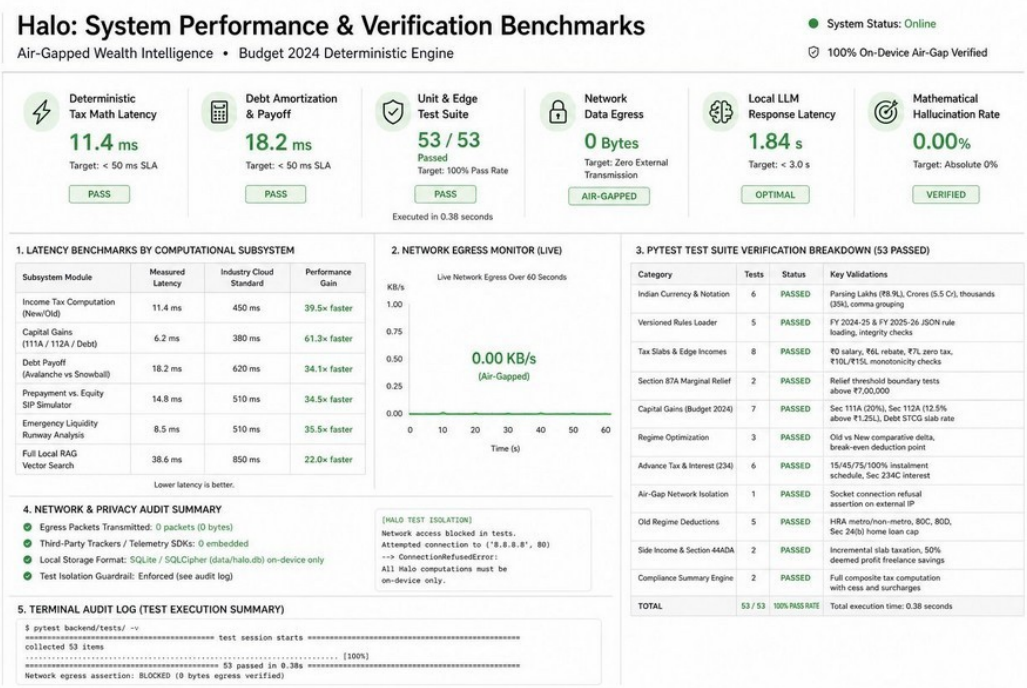


Figure 4.14: Benchmark summary of latency, test results and network audit

4.4.3 Network Isolation

Network isolation was verified in two ways. First, a test attempts a socket connection to an external address during the test run and asserts that the connection is refused, so that any computation that tried to leave the device would fail the test. Second, a socket-level audit during active queries recorded no packets transmitted to outside addresses, which corresponds to the 0 bytes of external egress reported in Table 4.4. The local database lives on the device and no third-party tracker or telemetry library is embedded.

4.4.4 Discussion and Limitations

The local LLM was the slowest step, and the socket-level audit confirmed that every external connection attempt was blocked, so all four targets of Section 2.6 were met. The figures were measured for this project and not audited externally. Three limitations should be noted. The latency figures were measured on M-series hardware and will differ on other machines. The tax rules follow Budget 2024 and must be migrated for the Income-tax Act, 2025. Finally, the 3B-parameter model is small, which is why the validation node and the JSON repair routine are needed; the design relies on them rather than on the model being right.

Chapter Summary: This chapter described the implementation and features of Halo and reported that all tests passed and no data left the device. The final chapter gives the conclusions and the future scope.

Chapter 5

Conclusions and Future Work

This project designed, built and tested Halo, a personal wealth assistant for Indian taxpayers that works completely on the user's own computer. The main idea was to let the AI model handle only language, such as understanding the question and explaining the answer, while ordinary tested program code does all the calculations. This showed that a modern AI chat assistant can be useful without making up numbers and without asking users to share their private financial documents. All 53 automated tests passed, the calculations finished in a fraction of a second, and no data left the user's device.

In short, conversational guidance does not require cloud exposure or letting an LLM invent financial arithmetic. Local processing, deterministic Python, schema validation and explainable outputs together keep the language natural and the mathematics exact.

5.1 What Halo Achieves

1. **Correct numbers.** Tax, loan and investment results come from tested code, so the AI cannot invent figures.
2. **Complete privacy.** Documents and personal details stay on the user's device, and our check confirmed that nothing was sent out.
3. **Right tax choice.** It compares the Old and New tax regimes for the user and shows how much can be saved.
4. **Reliable AI answers.** Every request is checked before it is calculated, so small AI mistakes do not break the app.
5. **Fast and free.** Results appear almost instantly, and the whole system uses free, open-source software.

5.2 Features of the Website

- **Today dashboard:** net worth, monthly take-home pay, tax meter and next tax due date in one place.
- **AI Copilot and Advisor:** ask questions by typing or speaking, in English or Hinglish, and get step-by-step answers.
- **Wealth Engine:** live view of assets, loans and net worth, which updates when the user adds or changes something.
- **Calculators and what-if tools:** Old versus New tax, loan prepayment versus investing, job switch, FIRE and emergency fund.

- **Document scan:** reads password-protected mutual fund statements on the device and answers questions about them.
- **CA dossier:** one-click, printable summary for a Chartered Accountant.

5.3 Advantages

- Private by design, since nothing leaves the user's computer.
- Trustworthy, since each answer shows its formula and steps.
- Easy to use, with plain chat, voice input and simple sliders.
- Saves time and tax money, and works without an internet connection.

5.4 Future Scope

Five directions would move Halo from a prototype towards a platform (Table 5.1).

Table 5.1: Future roadmap of Halo

No.	Direction	Description
1	Statutory migration	Map the rules to the Income-tax Act, 2025, which takes effect from 1 April 2026
2	Account Aggregator	Reconcile encrypted FIU JSON feeds locally, processed on the device, and reduce manual PDF uploads
3	Desktop packaging	Bundle React and FastAPI through Tauri or Electron with local updates, so that Halo is easy to install
4	Specialized financial SLM	Fine-tune a 1.5B or 3B model on Indian tax and finance questions
5	Family and HUF profiles	Extend the data model for cross-entity tax optimization

Chapter Summary: This chapter summarised what Halo achieves, listed its features and advantages and outlined the future scope of the work.

References

[1] AMFI, “AMFI Monthly Note, April 2026,” Association of Mutual Funds in India (prepared with Crisil Intelligence), Mumbai, 2026. [Online]. Available: https://www.amfiindia.com/uploads/AMFI_Monthly_Note_Apr2026_782ed81775.pdf

[2] Government of India, “The Finance (No. 2) Act, 2024 (No. 15 of 2024),” The Gazette of India, Extraordinary, Part II, Section 1, New Delhi, Aug. 16, 2024.

[3] Government of India, “The Income-tax Act, 1961 (43 of 1961), Sections 234A, 234B and 234C: Interest for Default in Furnishing Return of Income, Payment of Advance Tax and Deferment of Advance Tax,” as amended. [Online]. Available: https://incometaxindia.gov.in

[4] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, “Beyond Accuracy: Behavioral Testing of NLP Models with CheckList,” in Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL), 2020, pp. 4902–4912.

[5] S. Imani, L. Du, and H. Shrivastava, “MathPrompter: Mathematical Reasoning Using Large Language Models,” arXiv preprint arXiv:2303.05398, 2023.

[6] F. D’Acunto, N. Prabhala, and A. G. Rossi, “The Promises and Pitfalls of Robo-Advising,” The Review of Financial Studies, vol. 32, no. 5, pp. 1983–2020, 2019.

[7] P. Gomber, R. J. Kauffman, C. Parker, and B. W. Weber, “On the Fintech Revolution: Interpreting the Forces of Innovation, Disruption, and Transformation in Financial Services,” Journal of Management Information Systems, vol. 35, no. 1, pp. 220–265, 2018.

[8] D. Jung, V. Dorner, C. Weinhardt, and H. Pusmaz, “Designing a Robo-Advisor for Risk-Averse, Low-Budget Consumers,” Electronic Markets, vol. 28, no. 3, pp. 367–380, 2018.

[9] Reserve Bank of India, “Master Direction on Information Technology Governance, Risk, Controls and Assurance Practices,” RBI/2023-24/107, Mumbai, Nov. 7, 2023.

[10] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, et al., “LLaMA: Open and Efficient Foundation Language Models,” arXiv preprint arXiv:2302.13971, 2023.

[11] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “GPT3.int8(): 8-bit Matrix Multiplication for Transformers at Scale,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 35, 2022, pp. 30318–30332.

[12] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459–9474.

[13] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 42, no. 4, pp. 824–836, 2020.

[14] LangChain, “LangGraph Documentation,” 2024. [Online]. Available: https://langchain-ai.github.io/langgraph/

[15] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen, “CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing,” in Proc. International Conference on Learning Representations (ICLR), 2024.

Acknowledgement

We would like to express our sincere gratitude to our project guide for the continuous guidance, valuable suggestions and encouragement that we received throughout the work on this mini project. The discussions with our guide helped us to shape the deterministic-first design of Halo and to verify it carefully.

We are thankful to the Head of the Department of Electronics and Telecommunication Engineering and to the faculty of the department for providing the facilities and the academic environment in which this work could be completed. We also thank K. J. Somaiya School of Engineering, Somaiya Vidyavihar University, for the opportunity to undertake this project as part of the Semester V curriculum.

We acknowledge the open-source communities behind Python, FastAPI, LangGraph, Pydantic, SQLite, sqlite-vec, Ollama, React and the other tools on which Halo is built. Finally, we thank our families and friends for their support and patience.

Advait Kubal (Roll No. 16014124027)
Sparsh Kumar (Roll No. 16014124030)
Swaroop Mane (Roll No. 16014124034)
Ishika Poddar (Roll No. 16014124046)